Versão nativa Android do Waze Places — análise técnica

Documento interno para discussão com a equipe

---

1. "Versão nativa Android" pode significar 4 coisas — e cada uma muda a resposta

Antes de qualquer análise, é crítico definir o que se entende por "nativo", porque a resposta sobre backend muda dramaticamente:

Abordagem	Linguagem	Backend ainda necessário?
A. Kotlin/Java puro nativo	Kotlin + Jetpack Compose	**NÃO** (se quiser eliminar)
B. Cross-platform nativo (Flutter, KMM, React Native)	Dart / Kotlin / TypeScript	**NÃO** (mesma razão)
C. Wrapper de WebView (Capacitor, Cordova, TWA)	JS dentro de shell nativo	**SIM** — é a PWA empacotada
D. Híbrido com backend	Nativo + APIs nossas	Sim, por escolha

A pergunta interessante é A/B. C/D são variações da arquitetura atual com cara diferente.

---

2. Por que o backend existe HOJE (e por que isso muda no nativo)

A arquitetura atual tem o backend PHP por **duas razões fundamentais**:

Razão #1: CORS (Cross-Origin Resource Sharing)

Browsers proíbem JavaScript rodando em `places.wazebrasil.com` de fazer requests pra `www.waze.com/.../Issues/Search/List` com cookies, a menos que o servidor de destino mande headers tipo `Access-Control-Allow-Origin: https://places.wazebrasil.com` na resposta. **O Waze não manda** (porque a API deles é interna). Então a única saída é o backend agir de proxy: nosso PHP roda **fora do browser**, sem regras de CORS, hits o Waze como qualquer cliente HTTP, retorna pro frontend.

Em app nativo: CORS NÃO EXISTE. É uma proteção exclusiva de browser. Apps fazem HTTP request pra qualquer host, com qualquer cookie/header. Razão #1 evapora.

Razão #2: Segurança dos cookies

Cookies de sessão Waze (`_csrf_token` , `_web_session`) são credenciais de longo prazo (~28 dias [histórico: o valor do cookie morre em ~24–26 h sem renovação]). Se ficassem em `localStorage` do browser, qualquer XSS exfiltraria. Por isso encriptamos no backend (AES-256-CBC) e o client só guarda um `sessionToken` opaco — sem valor sozinho.

Em app nativo:

- Android tem **EncryptedSharedPreferences** + **Android Keystore** (TEE/Hardware-backed em devices modernos)
- Chaves de encriptação ficam no hardware seguro do device, fora do alcance até de root em alguns chips
- Sandboxing de apps Android é mais forte que sandboxing de origem de browser
- Não há XSS num app nativo (não tem `<script>` injetável)

Razão #2 também evapora — guardar os cookies localmente fica **mais seguro** num app nativo do que hoje no browser.

Conclusão direta: num nativo bem feito, **o backend pode ser totalmente eliminado**. App fala direto com Waze.

---

3. Fluxo de autenticação no nativo: 3 opções concretas

Aqui é onde fica interessante. Hoje, o user precisa exportar `cookies.txt` ou usar a extensão Chrome. Num nativo, dá pra fazer **muito melhor**:

Opção A: Login via WebView interna (RECOMENDADO)

```
1. App abre. User clica "Login"
2. Activity nova mostra WebView com https://www.waze.com/login
3. User loga normalmente (email/senha, 2FA se tiver, lembrar dispositivo, etc.)
4. WebView navega através do fluxo Waze
5. App detecta quando WebView chega em /editor (login concluído)
6. App pega cookies da WebView via CookieManager.getInstance().getCookie("waze.com")
7. App salva em EncryptedSharedPreferences
8. WebView fecha. App agora tem cookies pra usar
```

Pseudo-código Kotlin:

```kotlin
webView.webViewClient = object : WebViewClient() {
    override fun onPageFinished(view: WebView, url: String) {
        if (url.contains("/editor")) {
            val cookies = CookieManager.getInstance().getCookie("https://www.waze.com")
            cookieStore.save(cookies)  // EncryptedSharedPreferences
            finishLoginActivity()
        }
    }
}
webView.loadUrl("https://www.waze.com/login")
```

Vantagens:

- User loga **exatamente como sempre** (não precisa de extensão, não precisa exportar arquivo)
- Funciona com 2FA, captcha, lembrar dispositivo, "login com Google", tudo
- Cookies sempre frescos
- Zero passo manual de copiar/colar

Desvantagens:

- A primeira vez o user vê uma tela do Waze "dentro" do app — pode estranhar
- Se Waze mudar o domínio de redirect pós-login, app precisa atualizar

Opção B: Re-aproveitar cookies do Chrome instalado

Android tem `CustomTabsService` que abre links no Chrome instalado, **mantendo a sessão**. Mas o app **não consegue ler** os cookies do Chrome (sandboxing entre apps). Então isso resolve "user já tá logado no Waze pelo Chrome", mas não dá ao app acesso aos cookies.

Aplicação prática: zero. Descartar.

Opção C: Re-autenticação periódica programática

App armazena email/senha encriptados, faz login programático quando cookies expiram (preencher form via OkHttp). **NÃO RECOMENDADO** porque:

- Exige guardar senha do user (responsabilidade enorme)
- Waze tem bot detection / reCAPTCHA / device fingerprinting que pode pegar
- Viola Waze ToS de forma mais clara que o resto do app

---

4. O que se GANHA indo nativo

UX

- **Gestos de swipe verdadeiros**: hoje a PWA usa touch events no DOM com inércia simulada. Nativo tem `RecyclerView` + `ItemTouchHelper` ou Compose `swipeable` — feel muito melhor, especialmente com swipe-back animation, haptic feedback (`HapticFeedbackConstants`), spring animations naturais
- **Animações 60-120fps**: GPU-accelerated, sem competir com main thread JS
- **Splash screen nativa**: aparece em <100ms vs PWA esperando JS carregar
- **Status bar / nav bar customizadas**: cor que combina com o app, immersive mode
- **Modo paisagem** com layouts adaptativos otimizados
- **Foldables / tablets** com layouts diferenciados (Galaxy Fold etc.)

Background / Notificações

- **WorkManager**: pode pré-buscar PURs em background quando o user tá no WiFi à noite, deixar prontos pra triagem de manhã
- **Push notifications via FCM**: "10 novos PURs na sua área de gerência" — possivelmente o feature mais impactante pra retenção
- **Geofencing**: notificar quando o user passa próximo de um lugar com PUR pendente
- **Quick Settings tile**: botão "Triagem rápida" puxando direto na barra de notificações

Performance e tamanho

- App download: ~3-5MB inicial (Kotlin compactado) vs PWA total cacheada ~5MB+ (incluindo Tailwind JS bundle de 407KB)
- Updates incrementais via Play Store (só o que mudou)
- Sem service worker, sem cache hell, sem version skew
- Sem 3 camadas de cache pra manter sincronizadas

Segurança

- Cookies em hardware Keystore (TEE em chips modernos = quase impossível extrair sem comprometer o device)
- **Certificate pinning** pro `*.waze.com` (mesmo que CA seja comprometida, app não aceita cert errado)
- Sem XSS surface (não há `innerHTML`, não há `eval`)
- Sem CSP pra manter
- Sem `'unsafe-inline'` `'unsafe-eval'` (que o Tailwind JS força hoje)

Infraestrutura

- **Server cost: zero** se o backend sair. Hoje tem custo de hosting do PHP/Apache em algum lugar
- Sem `.htaccess`, sem `start.sh`, sem `PHP_CLI_SERVER_WORKERS=4`
- Sem encriptação `/tmp`, sem `SESSION_TTL`, sem sessões a limpar
- Bug rate cai muito (menos peças móveis = menos onde quebrar)

Distribuição

- Discoverability via Play Store search ("Waze places", "WME triagem")
- Auto-update gerenciado pela Play
- Editor compartilha link da Play Store em vez de URL

---

5. O que se PERDE indo nativo

iOS (público real)

- Android nativo = zero iPhone
- PWA hoje cobre iOS Safari (com limitações de install icon, mas funciona)
- iOS é minoria no Brasil (~20%), mas iOS é maioria em outros mercados de Waze editor ativo

Mitigação: usar **Flutter** ou **KMM (Kotlin Multiplatform Mobile)** pra compilar Android + iOS do mesmo código. Mas:

- Curva de aprendizado dobrada
- Apple Store é mais rigoroso que Play Store
- iOS pode rejeitar app que "scrape" Waze API (App Store Review Guidelines 5.2.5)

Desktop

- PWA funciona em Chrome, Edge, Firefox, Safari macOS
- Android nativo = só Android
- Quem edita Waze de PC (a maioria séria) perde a opção

Mitigação: manter PWA pra desktop + ter Android nativo pra mobile. Mas aí mantém DOIS códigos.

Velocidade de iteração

Aspecto	PWA	Android nativo
Bug crítico → correção em produção	minutos (`git push`)	24-72h (review Play Store)
Mudança visual simples	minutos	24-72h
Mudança que afeta API Waze	minutos (PHP só)	24-72h (todo user precisa atualizar app)
Reverter mudança ruim	minutos	24-72h

Pra projeto experimental ainda em iteração rápida (como o nosso, com dezenas de PRs em poucas semanas), isso é doloroso.

Curva de aprendizado

O time atual tem expertise em vanilla JS / PHP. Nativo Android exige:

- **Kotlin** (não muito difícil pra quem sabe JS, mas é outro paradigma)
- **Gradle** (build system notório por ser confuso)
- **Android SDK** (ciclo de vida de Activity, ViewModel, Compose, etc.)
- **Play Store policies** (data safety form, content rating, privacy policy obrigatória)
- **Conta Google Play Console** ($25 one-time)
- **Material Design** (se quiser parecer Android nativo)

Ou Flutter (Dart) que tem curva diferente.

Risco de banimento Waze (mesmo)

Vale notar: **o risco do Waze rejeitar/banir o app é o mesmo em ambos os casos.** Tanto a PWA quanto um app nativo são "uso não-oficial" da API interna deles. A diferença:

- PWA: Waze pode bloquear o IP do nosso servidor → matamos o app
- Nativo: Waze pode bloquear o User-Agent / certificate hash do app → mais difícil pra Waze fazer, mas possível
- Em ambos casos: Waze pode mudar a API e quebrar o app

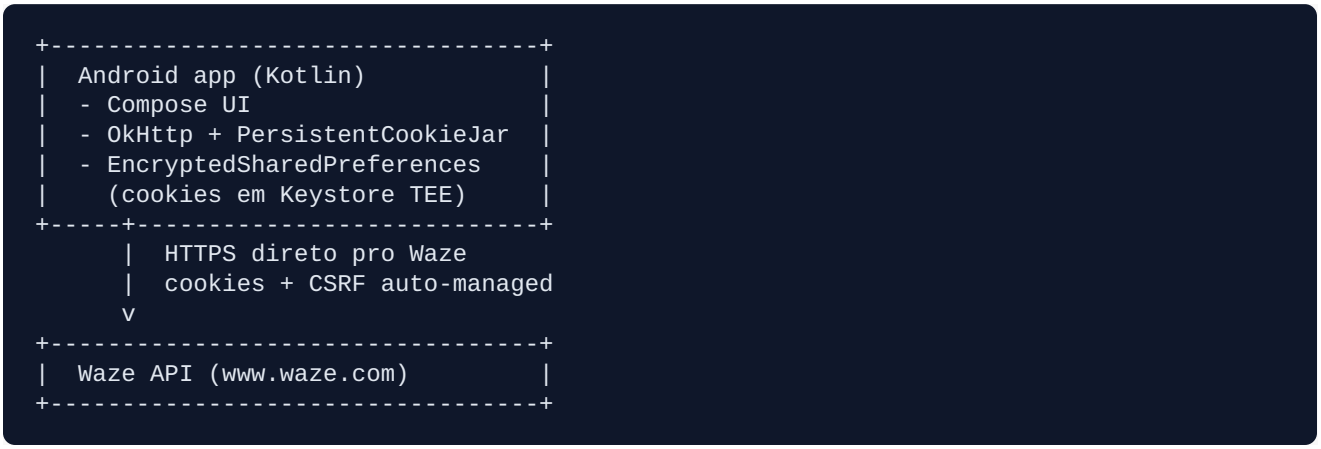
---

6. Arquitetura comparada visualmente

Hoje (PWA + backend)

```
+----------------------------------+
|  Browser (PWA)                   |
|  - JS no DOM                     |
|  - localStorage: sessionToken    |
|  - Service Worker (cache)        |
+-----+----------------------------+
      |  HTTPS POST /api/*.php
      |  body: {sessionToken, ...}
      v
+----------------------------------+
|  PHP backend (places.wazebrasil) |
|  - /tmp/sess_<hash> (AES-256)    |
|  - /tmp/waze_places.key (0600)   |
|  - Apache + .htaccess + CSP      |
+-----+----------------------------+
      |  cURL com cookies do user
      |  headers Referer/CSRF/UA
      v
+----------------------------------+
|  Waze API (www.waze.com)         |
+----------------------------------+
```

Nativo Android (sem backend)



1 hop a menos. 1 servidor a menos. ~1500 linhas de PHP a menos. Cookies sob proteção de hardware.

---

7. Esboço de implementação Kotlin

Pra ter ideia concreta de complexidade:

```kotlin
// 1. Storage de cookies
class CookieStore(context: Context) {
    private val masterKey = MasterKey.Builder(context)
        .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)
        .build()
    private val prefs = EncryptedSharedPreferences.create(
        context, "waze_cookies", masterKey,
        AES256_SIV, AES256_GCM
    )
    fun save(cookies: String) = prefs.edit().putString("cookies", cookies).apply()
    fun load() = prefs.getString("cookies", null)
    fun csrfToken() = load()?.let {
        Regex("_csrf_token=([^;\\s]+)").find(it)?.groupValues?.get(1)
    }
}

// 2. HTTP client com cookies persistidos
class WazeClient(private val store: CookieStore) {
    private val client = OkHttpClient.Builder()
        .cookieJar(object : CookieJar {
            override fun loadForRequest(url: HttpUrl) = parseCookies(store.load() ?: "")
            override fun saveFromResponse(url: HttpUrl, cookies: List<Cookie>) {
                store.save(cookies.joinToString("; ") { "${it.name}=${it.value}" })
            }
        })
        .build()

    suspend fun searchPlaces(filters: Filters) = withContext(Dispatchers.IO) {
        val body = Json.encodeToString(filters).toRequestBody("application/json".toMediaType())
        val req = Request.Builder()
            .url("https://www.waze.com/row-Descartes/app/v1/Issues/Search/List")
            .header("X-CSRF-Token", store.csrfToken() ?: error("no csrf"))
            .header("Referer", "https://www.waze.com/pt-BR/editor?env=row")
            .post(body)
            .build()
        client.newCall(req).execute().body!!.string()
            .let { Json.decodeFromString<SearchResponse>(it) }
    }
}

// 3. Login activity
class LoginActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            AndroidView(factory = { context ->
                WebView(context).apply {
                    settings.javaScriptEnabled = true
                    webViewClient = object : WebViewClient() {
                        override fun onPageFinished(view: WebView, url: String) {
                            if (url.contains("/editor")) {
                                val cookies = CookieManager.getInstance()
                                    .getCookie("https://www.waze.com") ?: ""
                                CookieStore(context).save(cookies)
                                finish()
                            }
                        }
                    }
                    loadUrl("https://www.waze.com/login")
                }
            })
        }
    }
}
```

Sem otimizações: ~150 linhas Kotlin pra ter login + cookies + uma chamada Waze funcionando. Substitui ~500 linhas PHP + ~300 linhas JS de auth/api/storage.

A UI de Tinder-cards seria mais código (Compose, animações, swipe), mas o **núcleo de comunicação com Waze fica brutalmente menor.**

---

8. Custo realista

Item	Hora estimada
Setup Android Studio + projeto básico Compose	2h
Auth flow WebView + storage encriptado	4-6h
Cliente Waze (OkHttp + tipos de resposta)	6-8h
UI tela principal (lista de cards, swipe)	8-12h
UI filtros (modal/screen)	4-6h
Stats, undo logic, dev mode	4-6h
Notificações FCM (opcional)	6-8h
Background prefetch (opcional)	4-6h
Polish, testes, edge cases	10-20h
Play Store setup (descrição, screenshots, privacy policy)	4-8h
Aprovação Play Store (espera)	1-3 dias
Total ativo	**~60-100h**

---

9. Caminhos intermediários (não-binários)

Não é tudo-ou-nada. Existem opções híbridas:

A. TWA (Trusted Web Activity)

- Wrapper Android ~50 linhas que abre a PWA em Chrome Custom Tab fullscreen
- Publica na Play Store
- Looks/feels quase como app nativo
- **NÃO elimina backend** (é a PWA por dentro)
- Custo: ~1 dia
- Ganho: discoverability + look nativo
- Perda: nenhuma técnica (PWA funciona igual)

B. PWA + Capacitor/Cordova

- Mesmo código JS rodando em WebView nativa
- Acesso a APIs nativas (notificações, câmera, geolocation forte)
- Pode publicar Play Store e App Store iOS
- **NÃO elimina backend** (a menos que reescreva acesso direto Waze via plugin nativo de fetch)
- Custo: 1-2 semanas
- Ganho: PWA com benefícios mobile, dual platform

C. Flutter (cross-platform real)

- Dart, código único pra Android + iOS
- Pode falar direto com Waze (sem backend)
- App nativo de verdade (não WebView)
- Custo: 80-150h (mais curva de Dart)
- Ganho: max alcance, max performance, sem backend
- Perda: ainda não tem desktop fácil (Flutter Desktop existe mas é nicho)

D. PWA + Android nativo coexistindo

- Mantém PWA pra desktop, novos editores experimentando
- Lança nativo Android pro caso "uso intenso mobile"
- Compartilha **nada** de código — duas bases separadas
- Custo: dobra a manutenção
- Ganho: cada plataforma com seu melhor
- Tradeoff: manter dois "produtos"

---

10. Recomendação franca

Pra o estado atual do projeto (iteração rápida, time sem experiência Android, audiência ~80% BR/mobile mas com cauda desktop importante):

Curto prazo (1-2 meses):

- **NÃO refazer nativo agora**. PWA cobre o caso.
- Focar em polir PWA pra mobile: install prompt funcional, swipe mais responsivo, talvez adicionar haptic feedback via Vibration API.
- TWA leva ~1 dia e dá "cara de app" + Play Store presence sem custo arquitetural.

Médio prazo (3-6 meses):

- Se mobile UX virar reclamação recorrente OU se quiser push notifications de PURs novos → **considerar Flutter**. Cobre Android + iOS, elimina backend, vale a invasão.
- Mantém PWA como "fallback web" pra desktop.

Longo prazo (1 ano+):

- Se app crescer a ponto de ter milhares de editores ativos diários, o **TWA + PWA backend** começa a custar (servidor PHP). Aí Flutter native compensa o investimento.

Não recomendo:

- Android Kotlin puro (perde iOS + desktop sem ganho proporcional)
- Migrar tudo de uma vez (alto risco, longa janela sem produto)

Resposta direta às perguntas chave

"Ainda precisaríamos do backend pra tratar cookies?" — em nativo de verdade (Kotlin, Flutter, KMM), **não**. App talks direto com Waze, cookies em Keystore. Backend pode ser deletado se TODAS as funcionalidades migrarem.

Exceção: se mantiver PWA + nativo coexistindo, backend continua existindo pra a PWA. Não pode ser deletado.

Bonus que não esperava: indo nativo, *muitos* problemas que viraram gotchas dolorosos (cache version skew, service worker, CSP, mod_pagespeed, Cloudflare cache) literalmente desaparecem. O app fica estruturalmente mais simples.

Documento gerado para discussão com a equipe do Waze Places — junho 2026